



Sistema de Gestión de Flota de Vehículos Eléctricos

SIGEFVE

Tecnologías de
Programación



Tabla de contenido

01

**Objetivo
general**

¿De qué trata el proyecto?

02

**Contexto
del negocio**

Información de lo que se
necesita

03

**Función del
lenguaje**

Distribución de
responsabilidades por lenguaje

04

Requisitos

Funcionalidades mínimas de
cada microservicio

Tabla de contenido

05

Arquitectura

¿Cómo es la arquitectura del sistema?

06

Diagramas de clase

Diagramas UML de clase de cada microservicio

07

Diagramas relacionales

Esquemas de bases de datos de cada microservicio

08

Demostración

Demostración de funcionamiento

01

Objetivo general



SIGEFVE



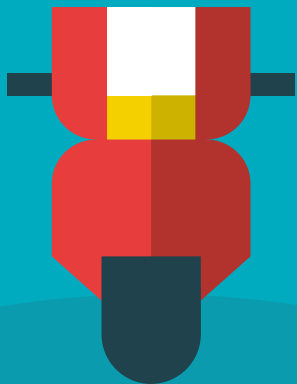
Objetivo general

Implementar un sistema aplicando correctamente el modelo de objetos integrando Java, Python y Go mediante arquitectura de microservicios.



02

Contexto del negocio



SIGEFVE

Contexto del negocio

Una empresa de reparto urbano tiene una flota de 15 vehículos eléctricos (vans, bicicletas eléctricas y motos).

Cada vehículo tiene:



Información de...

Telemetría

Nivel de batería, ubicación
GPS, temperatura del
motor, velocidad

Estado

Disponible, En Ruta, En
Mantenimiento,
Cargando

Ruta

Ruta asignada,
entregas pendientes,
entregas completadas



¿Qué debe hacer el sistema?

1

REGISTRO

De telemetría cada 15 segundos

2

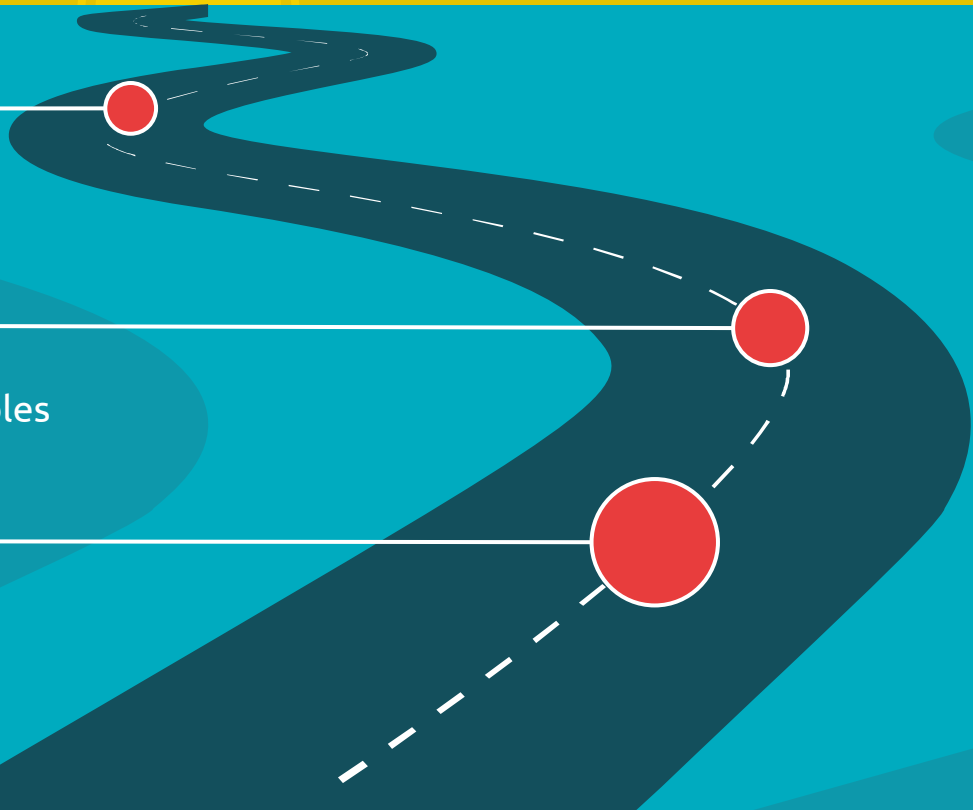
ASIGNAR

Rutas de entrega a vehículos disponibles

3

GENERAR

Alertas en ciertas condiciones



¿Qué debe hacer el sistema?

4

CALCULAR

Estadísticas de rendimiento de la flota

5

MOSTRAR

Estado actual y ubicación en dashboard web

Para esto, el sistema debe dividirse en microservicios independientes.

03

Responsa- bilidades



SIGEFVE

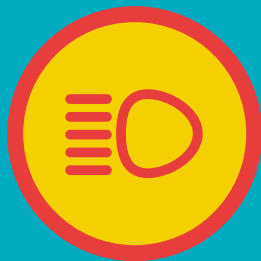


Responsabilidades por lenguaje



Java

Gestión de vehículos,
telemetría, rutas de
entrega



Python

Análisis de rendimiento,
sistema de alertas,
reportes

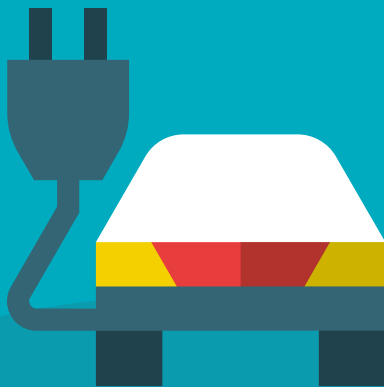


Go

API Gateway, autenticación,
enrutamiento

04

Requisitos



SIGEFVE

Funciones de Java



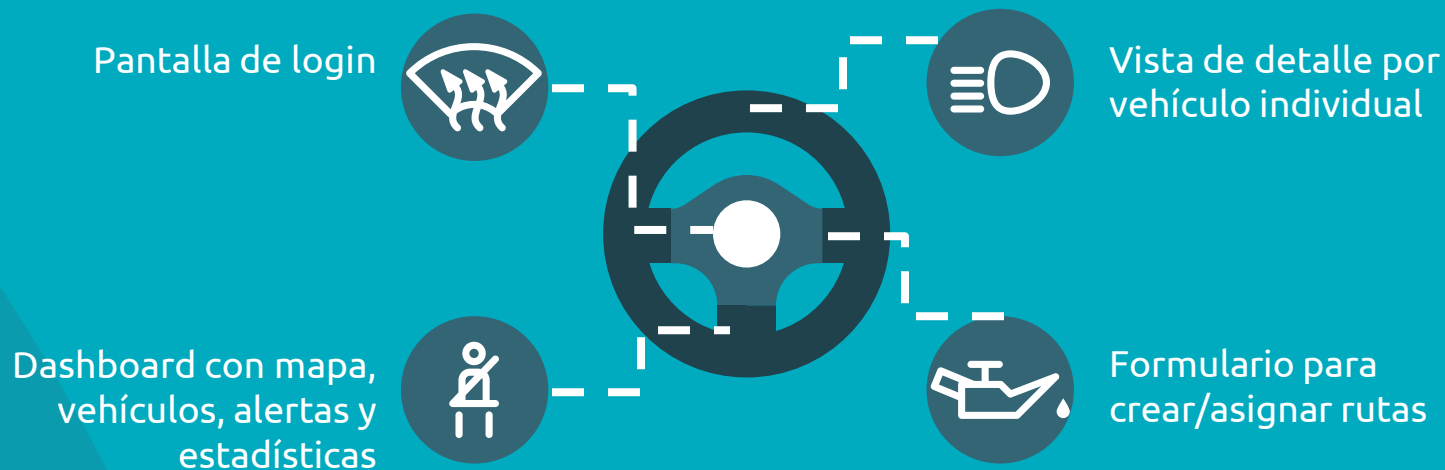
Funciones de Python



Funciones de Go

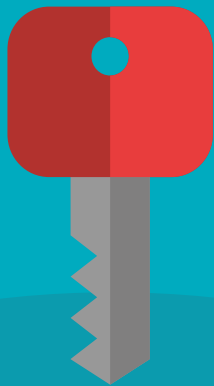


Funciones del frontend



05

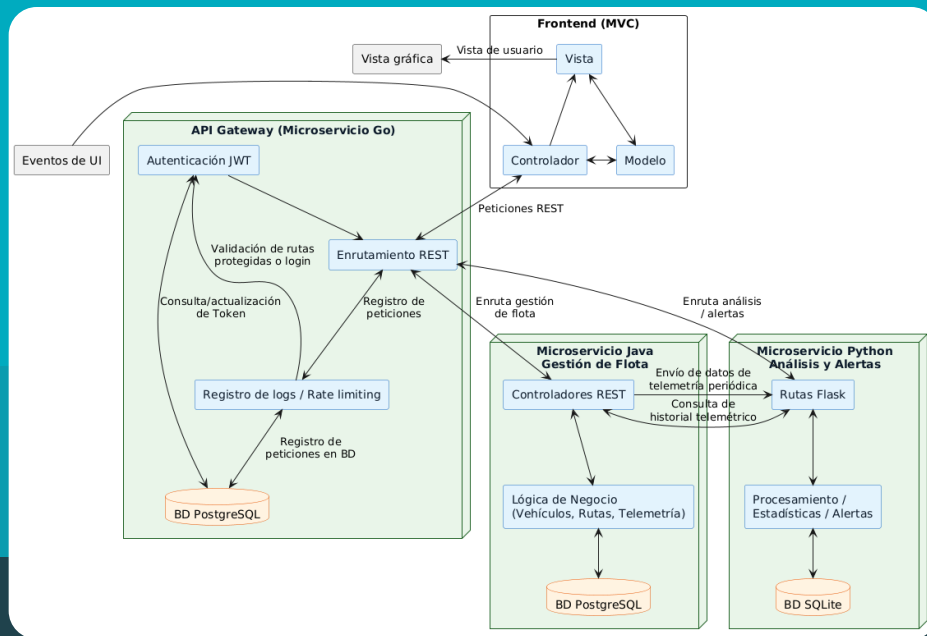
Arquitectura propuesta



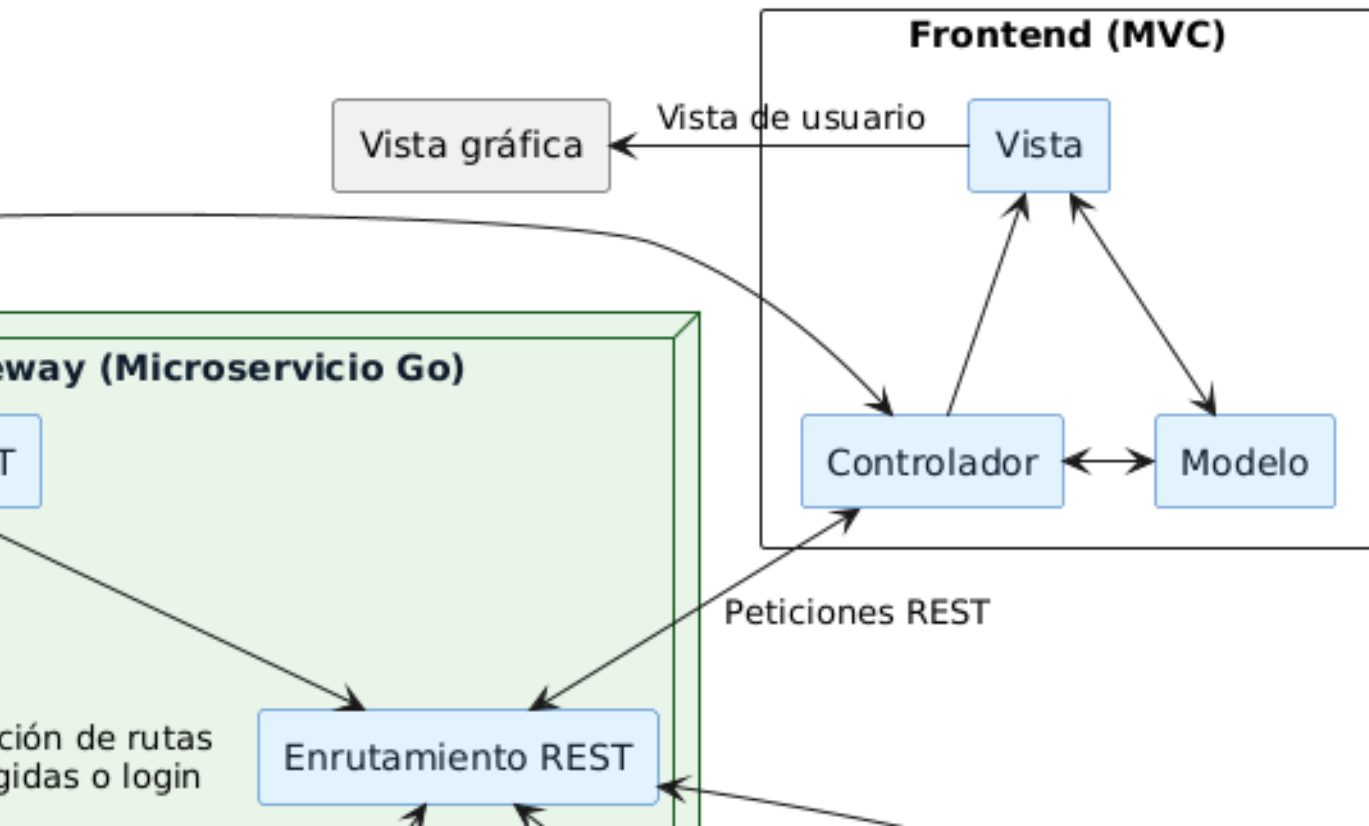
SIGEFVE

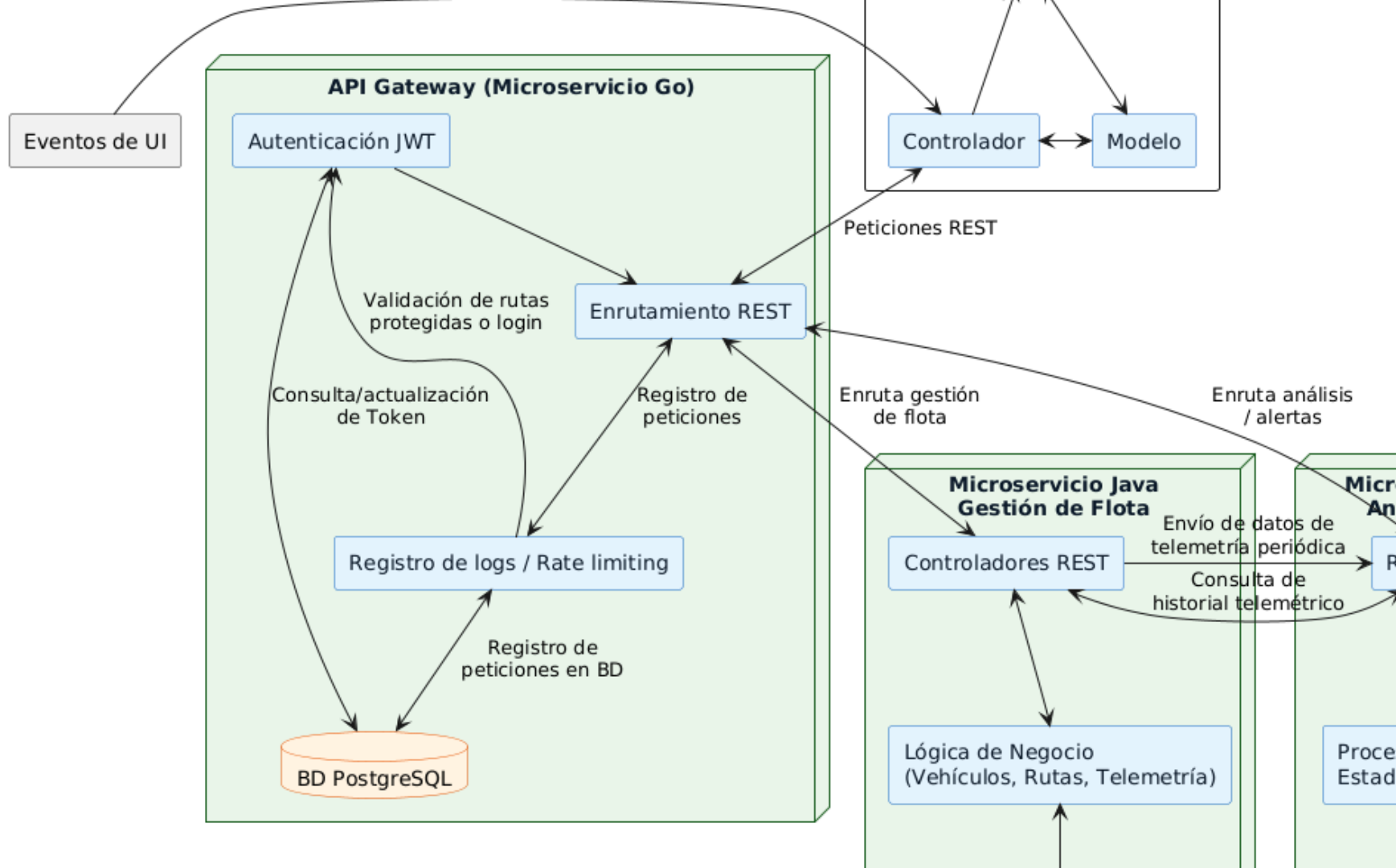


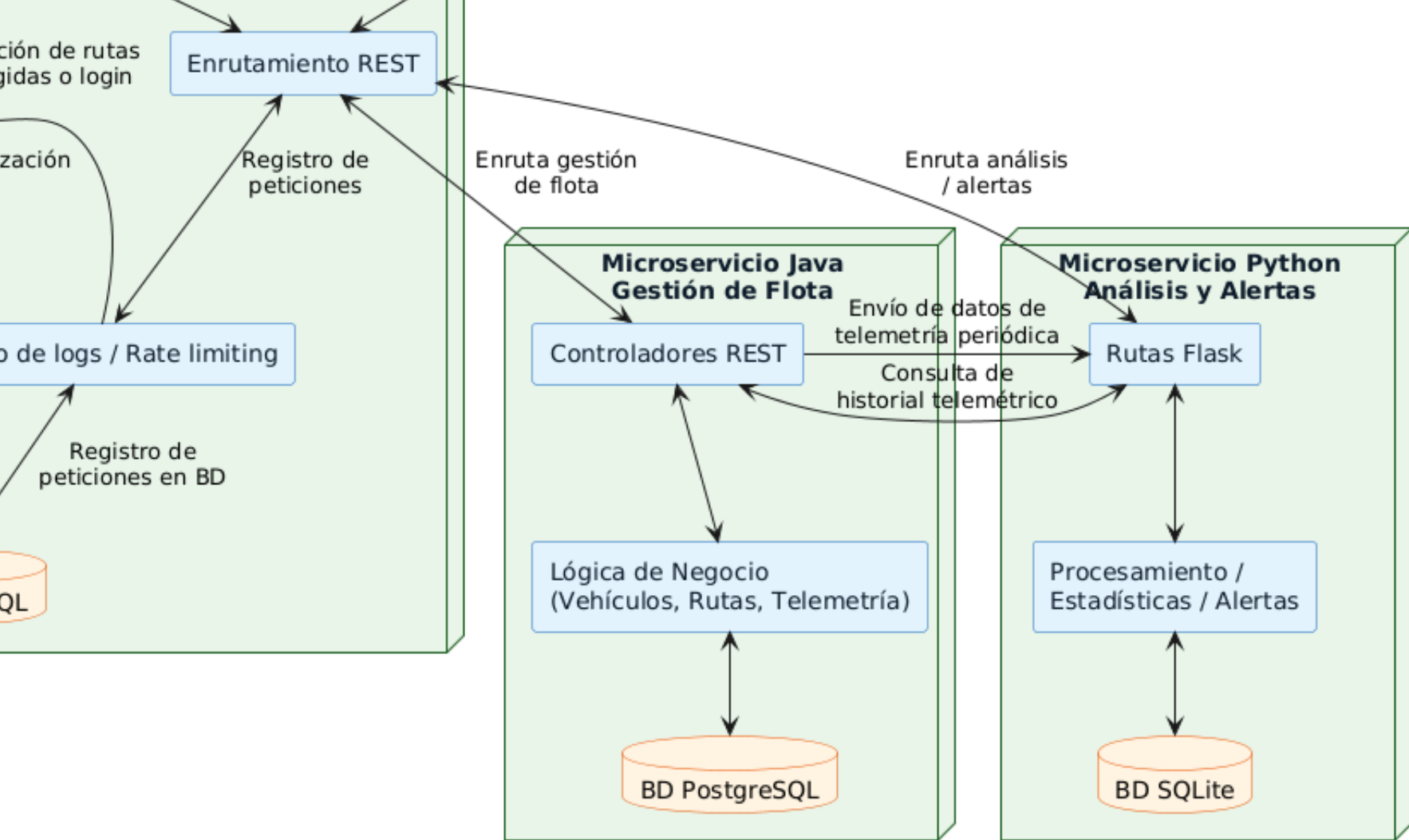
Arquitectura



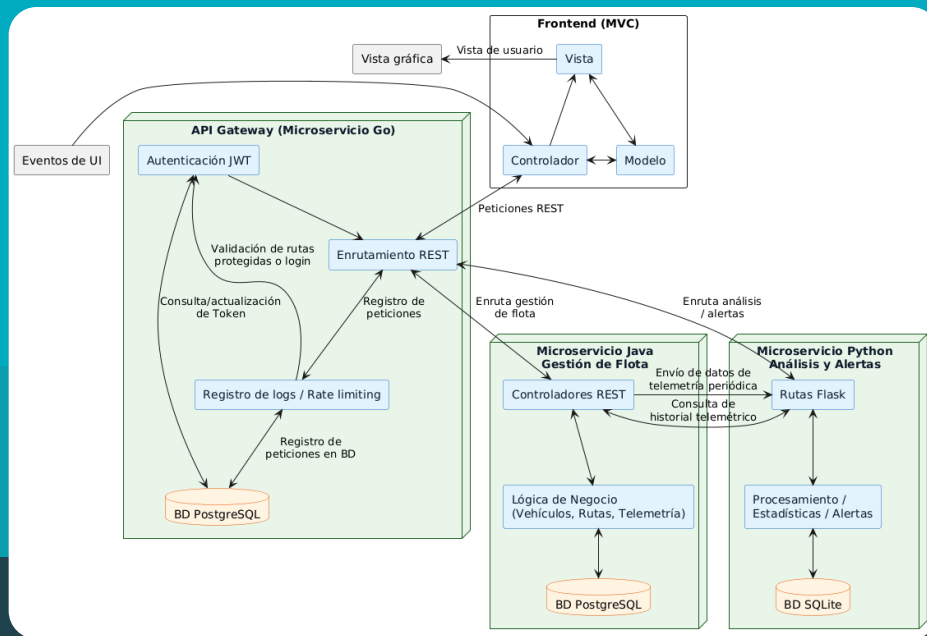
Arquitectura





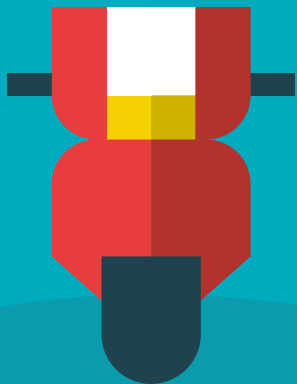


Arquitectura



06

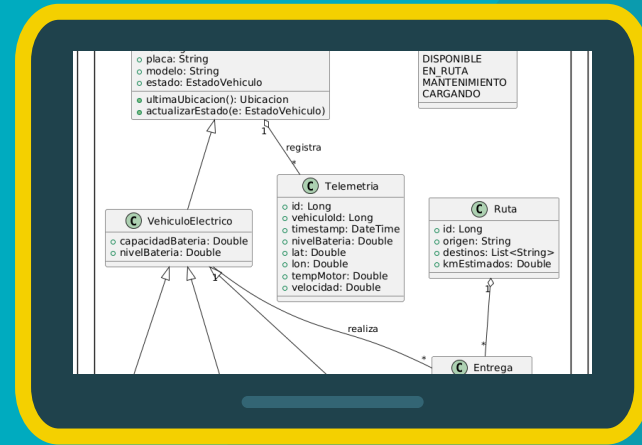
Diagramas de clase



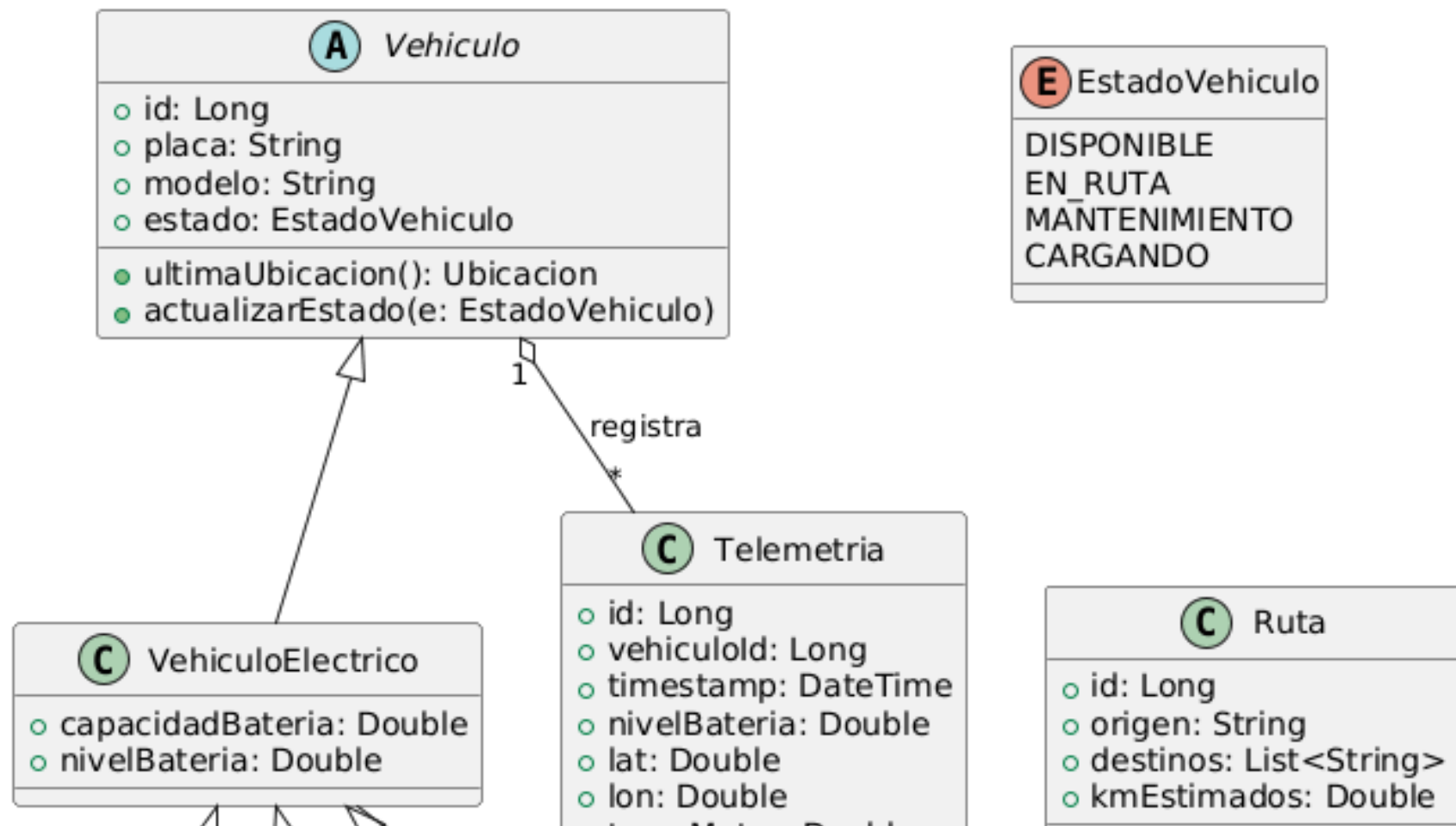
SIGEFVE

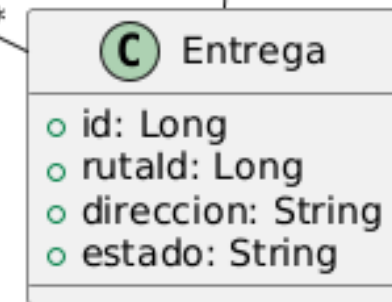
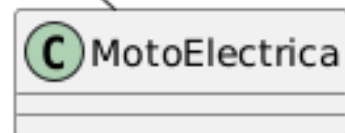
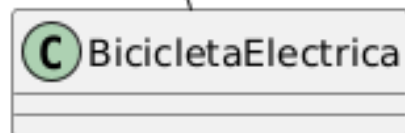
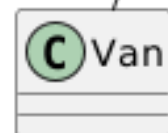
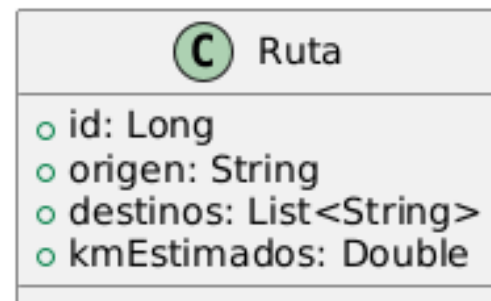
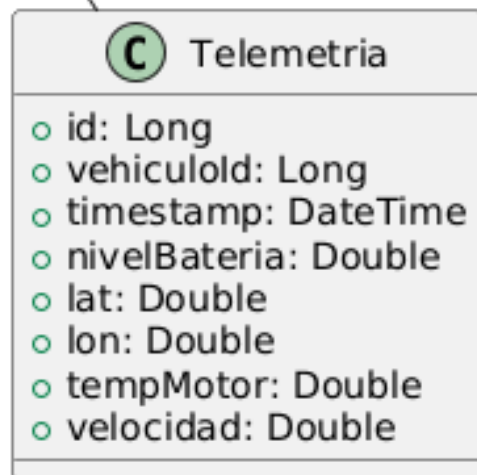
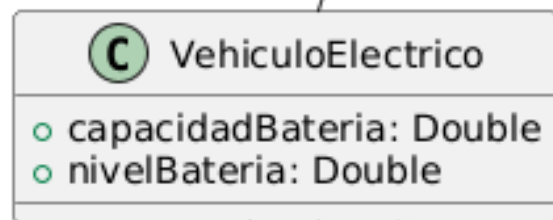
Diagrama de clases de Java

El diseño se basa en los principios de orientación a objetos, según los cuales la estructura del software debe modelar fielmente los objetos del dominio real (Booch, Rumbaugh & Jacobson, 1999).



model





realiza

1

*

*

Diagrama de clases de Java

El diseño se basa en los principios de orientación a objetos, según los cuales la estructura del software debe modelar fielmente los objetos del dominio real (Booch, Rumbaugh & Jacobson, 1999).

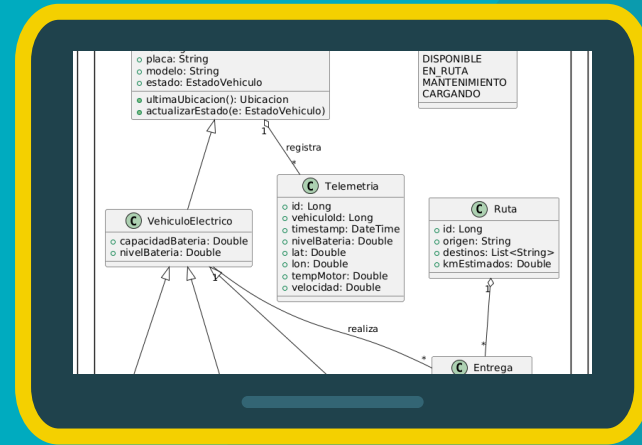
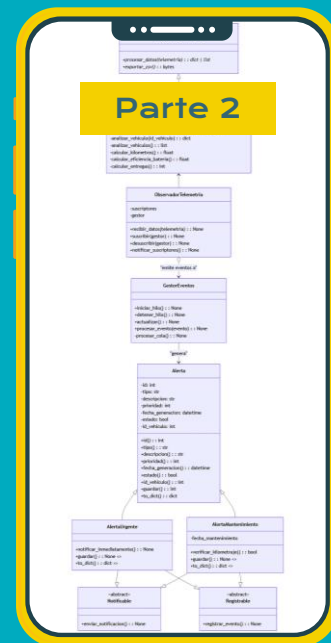
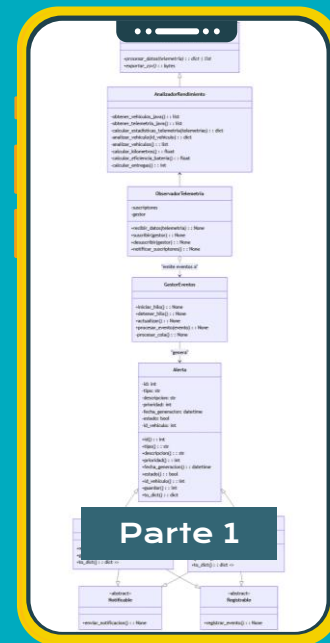
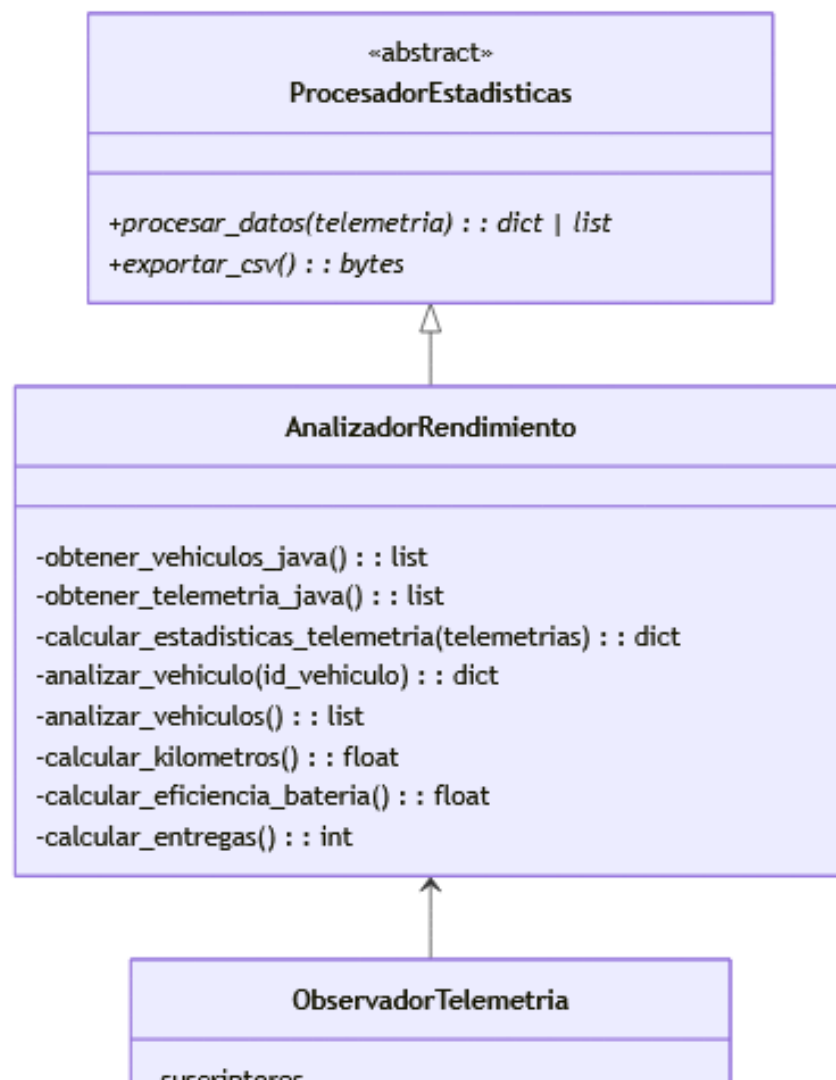


Diagrama de clases de Python

Modela el procesamiento de eventos y la generación de alertas implementando el patrón Observer, descrito por Gamma, Helm, Johnson y Vlissides (1995), para manejar la actualización automática entre productores y consumidores.





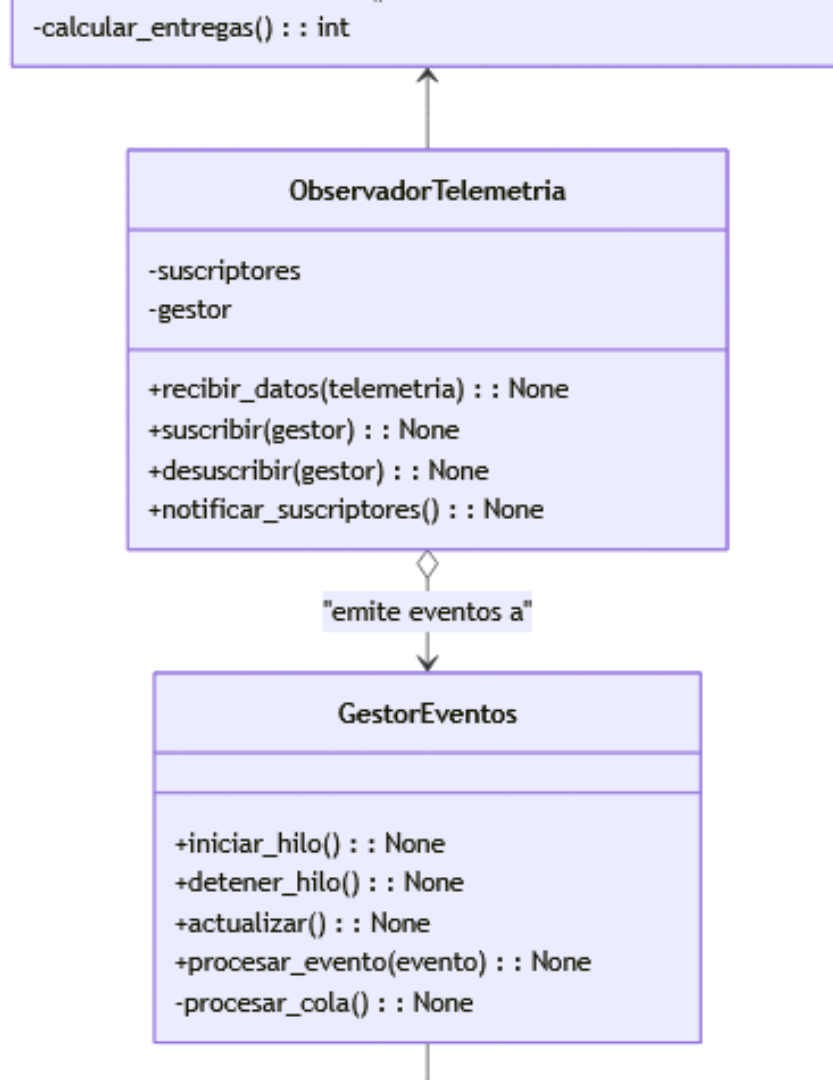
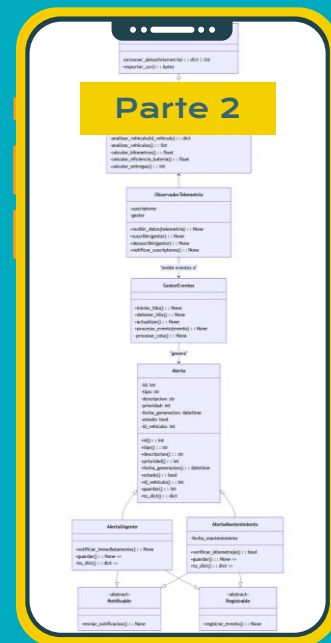
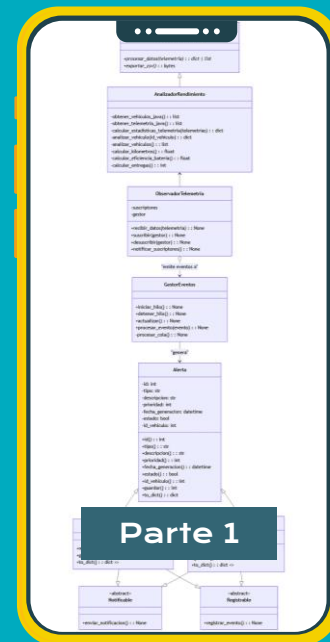


Diagrama de clases de Python

Modela el procesamiento de eventos y la generación de alertas implementando el patrón Observer, descrito por Gamma, Helm, Johnson y Vlissides (1995), para manejar la actualización automática entre productores y consumidores.



"genera"

Alerta

-id: int
-tipo: str
-descripcion: str
-prioridad: int
-fecha_generacion: datetime
-estado: bool
-id_vehiculo: int

+id() : : int
+tipo() : : str
+descripcion() : : str
+prioridad() : : int
+fecha_generacion() : : datetime
+estado() : : bool
+id_vehiculo() : : int
+guardar() : : int
+to_dict() : : dict

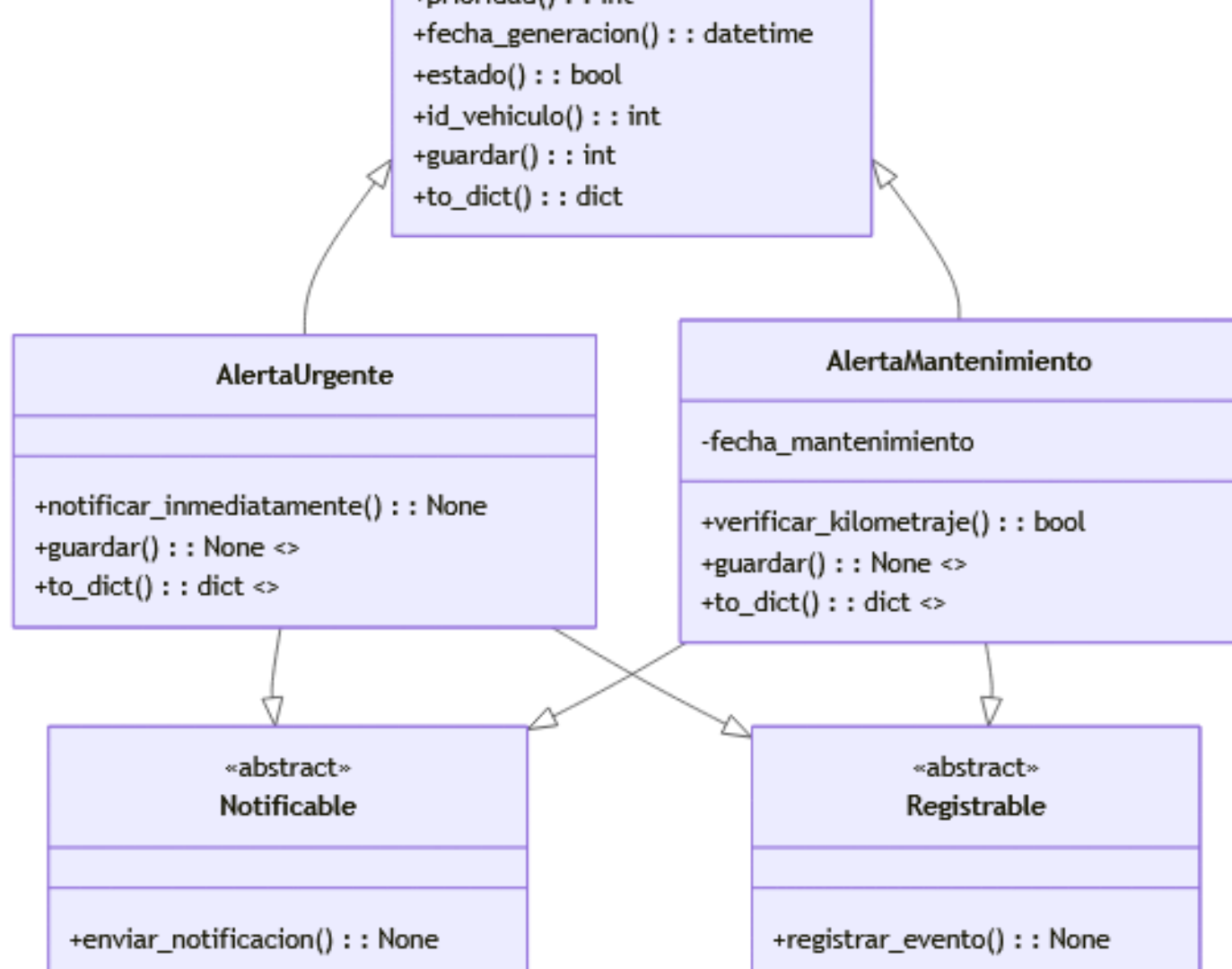
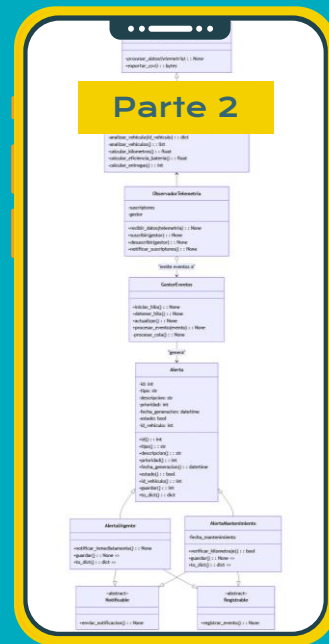
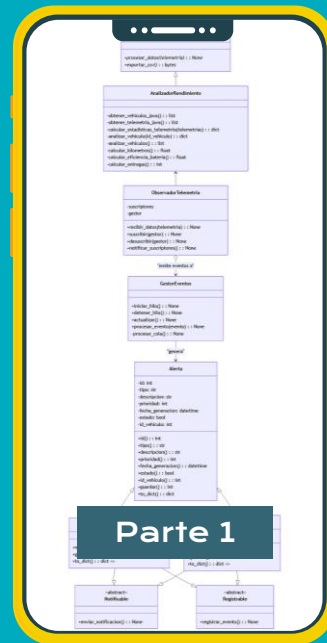


Diagrama de clases de Python

Modela el procesamiento de eventos y la generación de alertas implementando el patrón Observer, descrito por Gamma, Helm, Johnson y Vlissides (1995), para manejar la actualización automática entre productores y consumidores.



ucido)

Diagrama de tipos de Go

or

g, string): (Usuario, error)

en, error)

c

verificarContrasena(string, string): bool

string, error)

«struct»
InfoJWT

- Usuario string
- IDUsuario int
- Rol string
- jwt.RegisteredClaims

«struct»
Usuario

- ID: int
- Nombre: string
- Email: string
- ContraseñaHash: string
- Rol: string
- FechaCreacion: Time

«struct»
Token

- ID: int
- IDUsuario: int
- TokenJWT: string
- FechaEmision: Time
- FechaExpiracion: Time

«struct»
LogPetición

- ID: int
- IP: string
- Ruta: string
- Metodo: string
- Fecha: Time
- IDUsuario: *int

«struct»
LimitelP

- ID: int
- IP: string
- Conteo: int
- UltimaPetición: Time
- Bloqueado : bool

Diagrama de tipos de Go

«struct»
Autenticador

- RegistrarUsuario(string, string, string, string): (Usuario, error)
- Login(string, string): (Usuario, error)
- GenerarJWT(string, int, string): (Token, error)
- validarToken(string)
- RevocarToken(string): error
- JWTMiddleware(): gin.HandlerFunc
- AutorizarRol(string): gin.HandlerFunc

verificarContraseña(string, string): bool

hashContraseña(string): (string, error)

«struct»
InfoJWT

- Usuario string
- IDUsuario int
- Rol string
- jwt.RegisteredClaims

models

- ID: int
- Nombre string
- Email string
- Contraseña string
- Rol string
- FechaCreacion time.Time

Diagrama de tipos de Go

Módulo

auth

- Regis
- Login
- Gene
- valid
- Revo
- JWTM
- Autor

main

Ⓢ

«struct»
ServidorAPI

- BaseDatos: BaseDatos
- Router: Router
- IniciarServicio(): void

gateway

Ⓢ

«struct»
Router

- PrepararRutas(): *gin.Engine
- revisarSalud(*http.Client, string): string

ReverseProxy(string): ginHandlerFunc

database

Ⓢ

«struct»
BaseDatos

- Config: Config
- Conectar(): error
- Cerrar(): void
- Migraciones(): error

ObtenerLimitesIP(int): (LimitesIP, error)

ObtenerLogsPeticon (int): (LogPeticon, error)

Diagrama de tipos de Go

- JWTM
- Auton

database

Ⓢ

«struct»
BaseDatos

- Config: Config
- Conectar(): error
- Cerrar(): void
- Migraciones(): error

Ⓢ

«struct»
Config

- Host: string
- Puerto: int
- Usuario: string
- Contraseña: string
- Nombre: string
- SSLMode: string

○
ObtenerLimitesIP(int): (LimitIP, error)

○
ObtenerLogsPeticion (int): (LogPeticion, error)

07

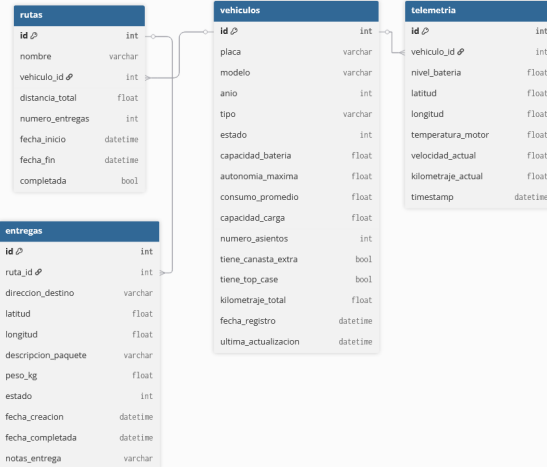
Diagramas relacionales

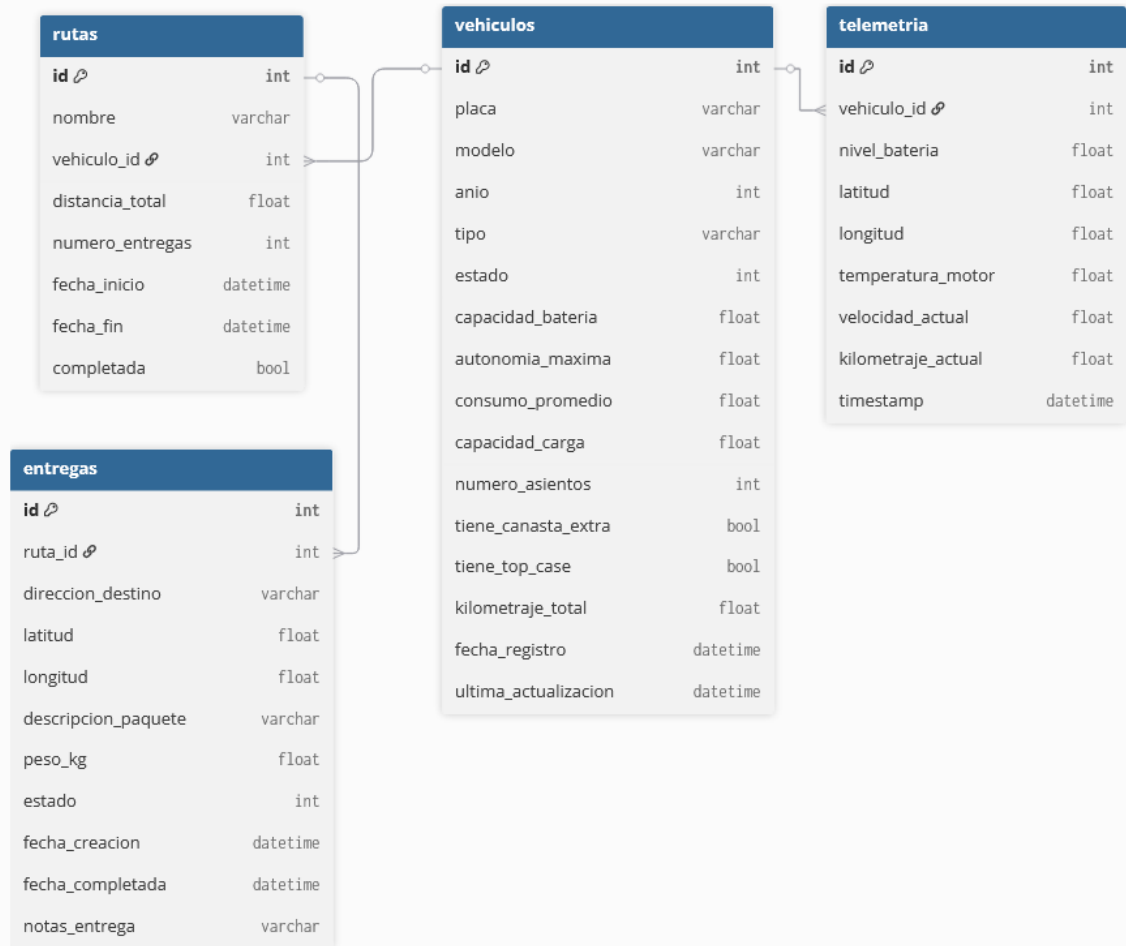


SIGEFVE

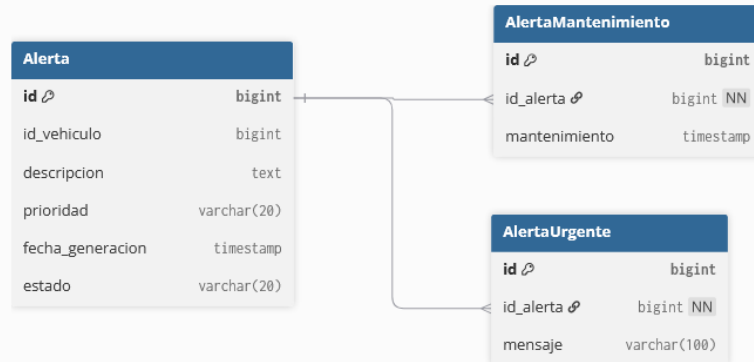


MICROSERVICIO JAVA





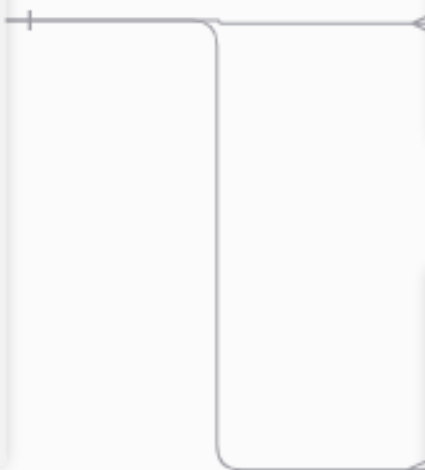
MICROSERVICIO PYTHON



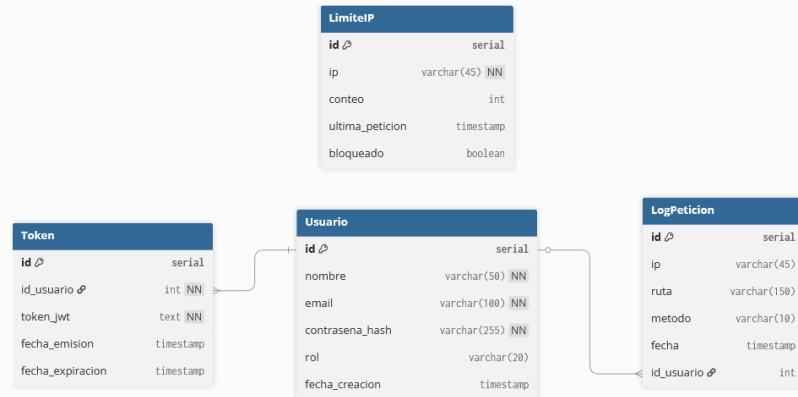
Alerta	
id 🔗	bigint
id_vehiculo	bigint
descripcion	text
prioridad	varchar(20)
fecha_generacion	timestamp
estado	varchar(20)

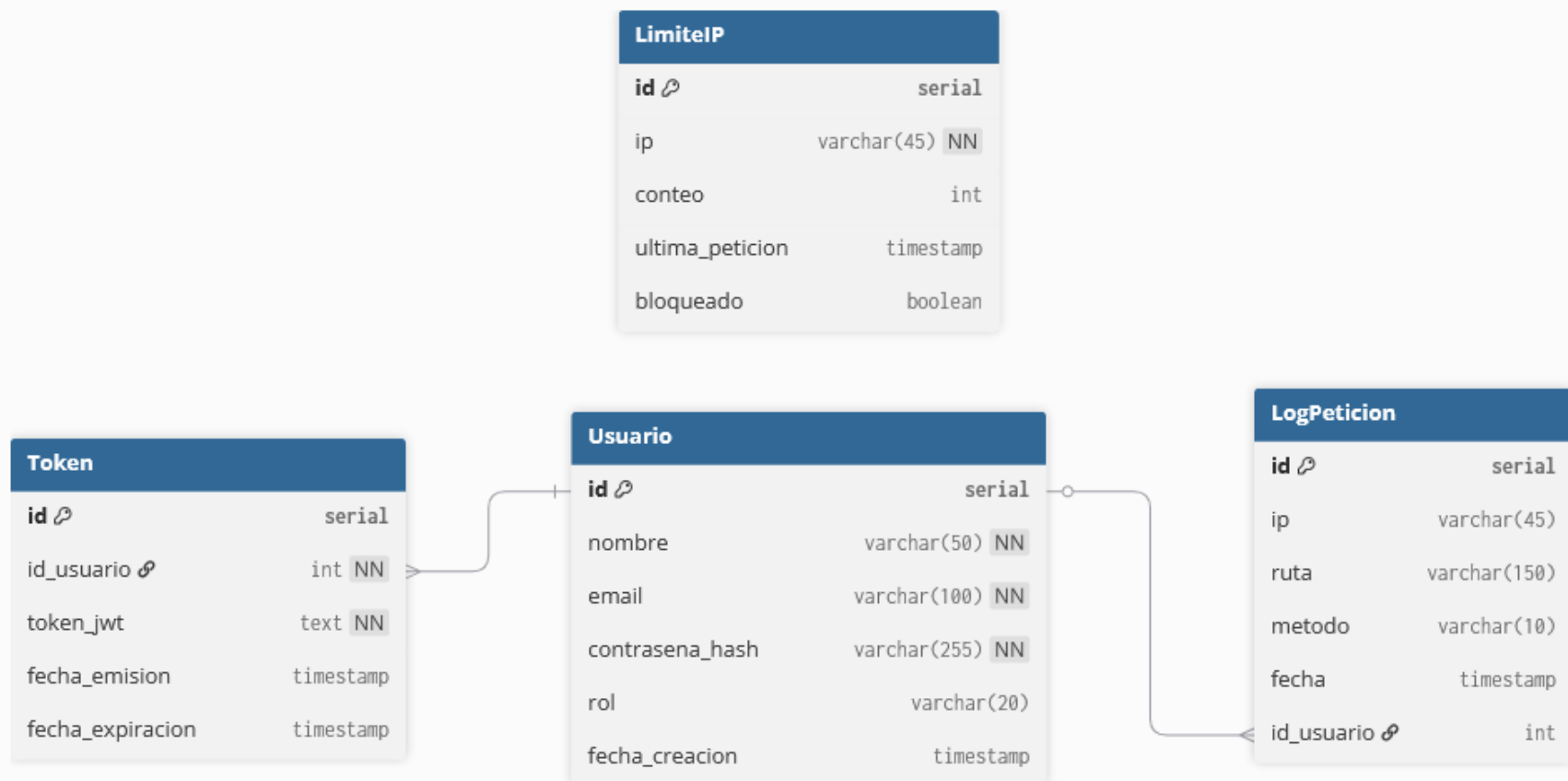
AlertaMantenimiento	
id 🔗	bigint
id_alerta 🔗	bigint NN
mantenimiento	timestamp

AlertaUrgente	
id 🔗	bigint
id_alerta 🔗	bigint NN
mensaje	varchar(100)

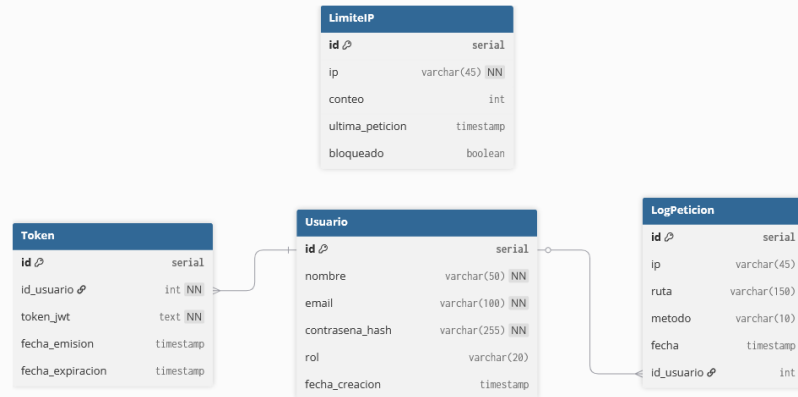


MICROSERVICIO GO



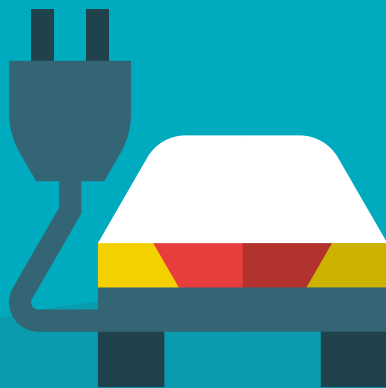


MICROSERVICIO GO



08

Demostración



SIGEFVE



Referencias



Guadaño, Y.
(2024). Patrones de
arquitectura de
microservicios: SAGA, API
Gateway y Service
Discovery. Paradigma.



Gamma, E., Helm, R.,
Johnson, R., & Vlissides, J.
(1995). Design Patterns:
Elements of Reusable
Object-Oriented Software.
Addison-Wesley.



Dashofy, E., Medvidović,
N., & Taylor, R. (2008).
Software Architecture:
Foundations, Theory, and
Practice. John Wiley &
Sons.

Referencias



Refactoring Guru. (s. f.)
Observer. Disponible en
[https://refactoring.guru/es/
/design-patterns/observer](https://refactoring.guru/es/design-patterns/observer)



Newman, S. (2015).
Building Microservices:
Designing Fine-Grained
Systems. O'Reilly Media.



Donovan, A. A., &
Kernighan, B. W. (2015).
The Go Programming
Language. Addison-
Wesley.

GRACIAS

¿Preguntas?

Aldo Iván López Vivanco
Jessica Hernández Mayahua
Coré Husaí Martínez Belmonte

CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, infographics & images by **Freepik** and illustrations by **Stories**

